AI WeatherWise

Deliver Intelligent Weather Forecasts and Insights

Project Demonstration

Team Details

Field	Details
Team ID	SWTID-2026-4935
Team Size	5
Team Leader	Rakshavi.V
Team Members	Raihana Fathima.S, Preetheeswari J, Priyanka S, Priyadarshini M
Tech Stack	Node.js, Express.js, MongoDB Atlas, Mongoose, JWT, bcryptjs, OpenWeatherMap, Google Gemini AI

Contents

1. Project at a glance
2. Demo scenario
3. Architecture in one picture
4. Live demo walkthrough
5. Demo script and talking points
6. Likely questions and answers
7. Outcome and future scope

1. Project at a Glance

AI WeatherWise is a RESTful backend that retrieves current weather for a location and uses Google Gemini AI to turn raw measurements into plain-language summaries and practical recommendations. Users register, log in, save favorite locations and request weather insights through secured APIs.

Area	What it delivers
Authentication	Registration and login with bcryptjs password hashing; JWT issued on login.
Favorite locations	Authenticated add, view, update and delete of saved cities (one user to many locations).
Weather	Current temperature, humidity, wind and conditions from OpenWeatherMap, with optional caching.
AI insights	Gemini-generated summary and event-planning recommendations.
Reliability	Validation, sanitization, centralized error handling and fallback when an external key is unavailable.

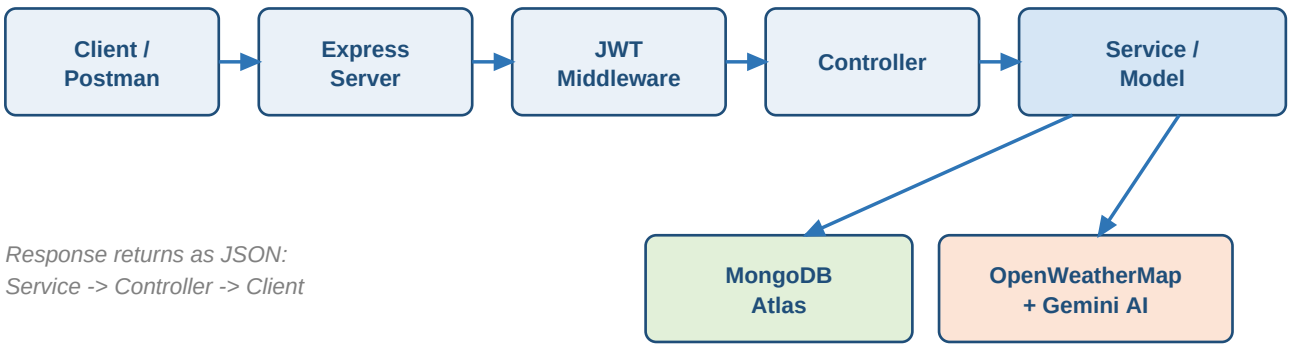
2. Demo Scenario

Priya is an event coordinator who organizes outdoor college functions, exhibitions, sports activities and cultural programs. Before finalizing a plan she needs quick, understandable weather information for each venue.

- Raw temperature, humidity and wind values are hard to interpret quickly.
- She manages several venues and needs them saved in one place.
- She needs advice such as "prepare an indoor alternative" when strong wind or rain is expected.

The demo follows Priya from registration to receiving an AI-generated recommendation.

3. Architecture in One Picture



The layered request path keeps responsibilities separate: routes map endpoints, middleware authenticates, controllers process requests, and services/models talk to MongoDB or the external weather and AI services.

4. Live Demo Walkthrough

Run each step in Postman against the local server. Replace the base URL and paths with your own if they differ.

Setup before the demo

```
npm install
npm start
# Expect: "MongoDB Connected: <host>" and server listening on PORT (e.g. 5001)
```

Step 1 - Register a user

Show that a new account is created and the password is stored only as a hash.

```
POST http://localhost:5001/api/auth/register
Content-Type: application/json

{ "name": "Priya", "email": "priya@example.com", "password": "Secret@123" }
```

Expected: success response, user created. **Point out:** password is never returned or stored in plain text.

Step 2 - Log in and get a JWT

```
POST http://localhost:5001/api/auth/login

{ "email": "priya@example.com", "password": "Secret@123" }
```

Expected: a JWT token. Copy it and set it as a Bearer token in the Authorization header for the next steps.

Step 3 - Show route protection

Call a protected endpoint without the token to show the 401 response, then repeat it with the token.

```
GET http://localhost:5001/api/favorites        (no token)
# Expected: 401 Unauthorized
```

Step 4 - Save a favorite location

```
POST http://localhost:5001/api/favorites
Authorization: Bearer <token>

{ "name": "College Ground", "city": "Chennai", "state": "Tamil Nadu", "country": "India" }
```

Then view, update and delete it to demonstrate full CRUD. **Point out:** each location is linked to the logged-in user only.

Step 5 - Retrieve current weather

```
GET http://localhost:5001/api/weather?city=Chennai
Authorization: Bearer <token>
```

Expected: temperature, humidity, wind speed and conditions. Repeat the request to show cached data can avoid another external API call.

Step 6 - Generate the AI recommendation

Use your weather recommendation endpoint (the path in your project may differ).

```
GET http://localhost:5001/api/ai/recommendation?city=Chennai
Authorization: Bearer <token>
```

Expected: a plain-language summary plus suggestions, for example preparing an indoor alternative if rain or strong wind is forecast, or proceeding outdoors if conditions are suitable.

Step 7 - Error handling (optional)

- Request weather for an invalid city: expect a clear error response.
- Remove the external API key from .env and show the fallback behavior, if enabled in your build.

Insert your Postman screenshots for Steps 1, 2, 5 and 6 below the matching step before presenting.

5. Demo Script and Talking Points

Time	Presenter focus	Key message
0:00 - 0:45	Introduce the team and problem	Event organizers struggle to interpret raw weather data for multiple venues.
0:45 - 1:30	Architecture slide	Three-tier design with MVC; Express, MongoDB Atlas, OpenWeatherMap and Gemini.
1:30 - 2:30	Register and login	bcryptjs hashing and JWT authentication.
2:30 - 3:30	Favorites CRUD and 401 test	Protected routes; users see only their own data.
3:30 - 4:30	Weather retrieval	Live data from OpenWeatherMap with caching.
4:30 - 5:30	AI recommendation	Gemini turns numbers into decisions for event planning.
5:30 - 6:00	Conclusion	Centralized, secure, modular backend that can be extended.

6. Likely Questions and Answers

Question	Suggested answer
Why MongoDB?	Flexible document storage suits users, locations and cached weather; Mongoose adds schemas and validation.
How is security handled?	bcryptjs hashed passwords, JWT-protected routes, secrets in .env, input validation and sanitization.
What if the weather or AI key is missing?	Selected operations use fallback functionality, and errors are returned as clear JSON responses.
How do you reduce API calls?	Weather responses and AI summaries can be cached in a weatherCache collection.
Why MVC for an API with no views?	Routes act as the view layer, controllers hold logic and models hold data, which keeps the code maintainable.
Can other providers be added?	Yes, weather and AI logic sit in separate services, so providers can be swapped or added.

7. Outcome and Future Scope

AI WeatherWise helps event organizers prepare with confidence by converting complex weather data into simple, actionable information, all through one secured backend.

- React dashboard with visual weather cards and AI recommendation panels.
- Multi-day forecasts and weather alerts for saved locations.
- Additional weather providers and AI models.
- Admin tools for user management, monitoring and backups.

Resources

Item	Link
API testing video	API TESTING .mp4 (add link)
Demo video	Backend demo.mp4 (add link)
Source code	Code (add GitHub/Drive link)